梯度AI标准化结构指南

> *一键初始化 AI 辅助开发工作区的标准化结构，让 AI Agent 更高效地为你工作。*

前置条件

在开始之前，请确保你的环境满足以下要求：

条件	最低版本	检查命令
Node.js	>= 18	`node -v`
Git	任意	`git --version`
Cursor IDE	最新版	打开 Cursor 即可

如果还没有安装 Node.js，请前往 https://nodejs.org/ 下载安装。 如果还没有安装 Git，请前往 https://git-scm.com/ 下载安装。

第一步：安装 Skill 到项目

打开终端，进入你的项目根目录，执行：

```
npx @tdio/skills add git@git.tiduyun.com:fe-infra/td-skills.git --skill
init-ai-workspace -y
```

> ***可选：*** *如果你经常使用该仓库的 Skills，可以先注册仓库，后续安装更方便：*
>
> ```
> # 注册仓库（只需一次）
> npx @tdio/skills repos add git@git.tiduyun.com:fe-infra/td-skills.git --
> clone
> # 之后可以用短名安装
> npx @tdio/skills add td-skills/init-ai-workspace
> ```

安装完成后，你的项目中会出现：

```
你的项目/
└── .cursor/
    └── skills/
        └── init-ai-workspace/    ← 已安装
```

第二步：在 Cursor 中执行初始化

1. 打开 Cursor IDE，确保当前打开的是你的项目
2. 切换到 **Agent 模式**（聊天输入框左侧的模式切换按钮）
3. 在聊天输入框中输入：

```
init workspace
```

4. 按回车发送

第三步：跟随 Agent 引导完成初始化

Agent 会用中文和你交互，整个过程分为 5 步：

3.1 结构介绍

Agent 先向你展示将要创建的工作区结构概览，让你了解每个部分的用途。

3.2 回答几个问题

Agent 会依次问你：

- **Q1**：是否需要「前端开发规范」或「后端开发规范」Skill？（可选）
- **Q2**：是否需要「经验沉淀」目录和「FAQ」目录？（推荐但可选）
- **Q3**：你的项目名称、技术栈、主要模块目录是什么？

> *根据你的回答，Agent 会决定创建哪些内容。*

3.3 自动执行初始化脚本

Agent 根据你的选择运行初始化脚本，自动完成：

- 从远程仓库安装 6 个必须 Skills（已有的会更新，没有的会拉取）
- 复制 4 个快捷命令到 `.cursor/commands/`
- 创建必须的文档目录

3.4 生成 AGENTS.md

Agent 根据你提供的项目信息，生成一份项目级的 `AGENTS.md` 指南文件。这是 AI Agent 了解你项目的「说明书」。

3.5 完成提示

Agent 告诉你初始化结果，以及后续可以做的事。

初始化后的目录结构

完整初始化后，你的项目会新增以下结构（`*` 标记为可选项）：

```
你的项目/
├── AGENTS.md                          # AI Agent 项目指南
├── .cursor/
│   ├── skills/                        # Agent 技能库
│   │   ├── brainstorming/             #   需求探索与方案脑暴
│   │   ├── skill-loader/              #   自动加载匹配的 Skill
│   │   ├── update-codemaps/           #   更新代码架构地图
│   │   ├── writing-plans/             #   编写实现计划
```

```
│   │   ├── writing-skills/           #   创建/编辑 Skill
│   │   ├── writing-specs/            #   编写需求/设计/任务规格文档
│   │   ├── frontend-conventions/ *   #   前端开发规范（可选）
│   │   └── backend-conventions/  *   #   后端开发规范（可选）
│   └── commands/                     # 快捷命令
│       ├── faq.md                    #   /faq — 维护 FAQ 知识库
│       ├── learn.md                  #   /learn — 提取可复用经验
│       ├── task-alignment-review.md  #   /task-alignment-review — 规格对齐审查
│       └── update-codemaps.md        #   /update-codemaps — 刷新代码地图
├── docs/
│   ├── codemaps/                     # 代码架构地图（高层概览）
│   ├── plans/                        # 实现计划文档
│   ├── specs/                        # 需求/设计/任务规格文档
│   └── learned/ *                    # 经验沉淀（可选）
└── FAQ/ *                            # 常见问题库（可选）
```

各部分说明

AGENTS.md

AI Agent 的「项目说明书」。Agent 在每次对话开始时会读取它，了解你的项目结构、技术栈、开发规范和测试约定。**项目必备文件。**

.cursor/skills/ — Agent 技能库

每个 Skill 是一个文件夹，里面有一个 `SKILL.md` 文件，教 Agent 如何完成特定类型的任务。

Skill	作用	什么时候触发
`brainstorming`	引导需求探索、方案对比	你说「帮我想想怎么做 XX」
`skill-loader`	自动识别并加载相关 Skill	每次对话开始时自动触发
`update-codemaps`	扫描项目结构，更新架构文档	你说「更新代码地图」
`writing-plans`	生成可执行的实现计划	你说「帮我做个计划」
`writing-skills`	创建/编辑新的 Skill	你说「帮我写个 Skill」
`writing-specs`	生成需求→设计→任务三阶段规格	你说「帮我写需求文档」

.cursor/commands/ — 快捷命令

在 Cursor 聊天框输入 `/命令名` 即可触发，相当于快捷指令：

命令	作用
`/faq`	将当前问题/解决方案录入 FAQ 知识库
`/learn`	从当前会话提取可复用的经验模式

`/task-alignment-review`	对照规格文档检查实现是否对齐
`/update-codemaps`	刷新项目代码架构地图

docs/ — 文档目录

目录	作用
`docs/codemaps/`	代码架构地图 — 项目的高层结构概览，帮 Agent 快速理解全局
`docs/plans/`	实现计划 — 功能开发前的执行步骤规划
`docs/specs/`	规格文档 — 正式的需求/设计/任务规范，每个功能一个文件夹
`docs/learned/`	经验沉淀 — AI 会话中发现的可复用模式，避免重复踩坑

FAQ/

团队共享的常见问题知识库。用 `/faq` 命令添加条目，自动维护索引。

初始化后的日常使用

开发新功能的推荐流程

```
脑暴想法 → 编写规格 → 制定计划 → 执行开发 → 审查对齐
```

对应的 Agent 交互：

1. 「帮我想想怎么实现 XX 功能」→ 触发 `brainstorming`
2. 「帮我写需求文档」→ 触发 `writing-specs`
3. 「帮我做个实现计划」→ 触发 `writing-plans`
4. 开始编码，Agent 自动加载项目规范
5. `/task-alignment-review` → 检查实现是否符合规格

日常维护

```
# 检查 Skills 是否有更新
npx @tdio/skills check

# 更新所有 Skills 到最新版
npx @tdio/skills update
```

常见问题

Q：初始化时提示 "npx 未找到" 怎么办？ A：请安装 Node.js >= 18，安装后重启终端。

Q：我的项目已经有部分目录了，会被覆盖吗？ A：不会。脚本遇到已存在的文件会跳过，不会覆盖你的内容。Skills 如果已存在则会更新到最新版。

Q：AGENTS.md 已经有了，会被覆盖吗？ A：不会自动覆盖。Agent 会问你要覆盖、合并还是跳过。

Q：我只想更新 Skills，不想重新初始化怎么办？ A：直接运行 `npx @tdio/skills check && npx @tdio/skills update` 。

Q：Windows 系统能用吗？ A：可以。脚本使用 Node.js 编写，Windows / macOS / Linux 全平台支持。